

Домашняя страница / Мои курсы / Факультет физико-математических и естественных наук
/ 09.00.00. Информатика и вычислительная техника / 09.03.03 Прикладная информатика (бакалавриат)
/ Архитектура компьютеров и операционные системы. Раздел "Операционные системы" (09.03.03, НПИбд) / Лабораторная работа № 2
/ Лабораторная работа № 2

Архитектура компьютеров и операционные системы. Раздел "Операционные системы" (09.03.03, НПИбд)

Лабораторная работа № 2

1. [Цель работы](#)
2. [Теоретические сведения](#)
 1. [Системы контроля версий. Общие понятия](#)
 2. [Примеры использования git](#)
 1. [Основные команды git](#)
 2. [Стандартные процедуры работы при наличии центрального репозитория](#)
 3. [Работа с локальным репозиторием](#)
 4. [Работа с сервером репозитория](#)
 3. [Базовая настройка git](#)
 1. [Первичная настройка параметров git](#)
 2. [Учёт переносов строк](#)
 4. [Создание ключа ssh](#)
 1. [Общая информация](#)
 2. [Создание ключа ssh](#)

3. [Добавление SSH-ключа в учётную запись GitHub](#)
5. [Верификация коммитов с помощью PGP](#)
 1. [Общая информация](#)
 2. [Создание ключа](#)
 3. [Экспорт ключа](#)
 4. [Добавление PGP ключа в GitHub](#)
 5. [Подписывание коммитов git](#)
 6. [Настройка автоматических подписей коммитов git](#)
6. [Проверка коммитов в Git](#)
 1. [Режим бдительности \(vigilant mode\)](#)
3. [Задание](#)
4. [Последовательность выполнения работы](#)
 1. [Установка программного обеспечения](#)
 1. [Установка git](#)
 2. [Установка gh](#)
 2. [Базовая настройка git](#)
 3. [Создайте ключи ssh](#)
 4. [Создайте ключи pgp](#)
 5. [Настройка github](#)
 6. [Добавление PGP ключа в GitHub](#)
 7. [Настройка автоматических подписей коммитов git](#)
 8. [Настройка gh](#)
 9. [Шаблон для рабочего пространства](#)
 1. [Создание репозитория курса на основе шаблона](#)
 2. [Настройка каталога курса](#)
5. [Содержание отчёта](#)
6. [Контрольные вопросы](#)

Лабораторная работа *Первоначальная настройка git.*

Цель работы

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

Теоретические сведения

Системы контроля версий. Общие понятия

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

Примеры использования git

- Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды `git` с различными опциями.
- Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией.

Основные команды git

- Перечислим наиболее часто используемые команды `git`.
- Создание основного дерева репозитория:

```
git init
```

- Получение обновлений (изменений) текущего дерева из центрального репозитория:

```
git pull
```

- Отправка всех произведённых изменений локального дерева в центральный репозиторий:

```
git push
```

- Просмотр списка изменённых файлов в текущей директории:

```
git status
```

- Просмотр текущих изменений:

```
git diff
```

- Сохранение текущих изменений:

- добавить все изменённые и/или созданные файлы и/или каталоги:

```
git add .
```

- добавить конкретные изменённые и/или созданные файлы и/или каталоги:

```
git add имена_файлов
```

- удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории):

```
git rm имена_файлов
```

- Сохранение добавленных изменений:

- сохранить все добавленные изменения и все изменённые файлы:

```
git commit -am 'Описание коммита'
```

- сохранить добавленные изменения с внесением комментария через встроенный редактор:

```
git commit
```

- создание новой ветки, базирующейся на текущей:

```
git checkout -b имя_ветки
```

- переключение на некоторую ветку:

```
git checkout имя_ветки
```

- (при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой)
- отправка изменений конкретной ветки в центральный репозиторий:

```
git push origin имя_ветки
```

- слияние ветки с текущим деревом:

```
git merge --no-ff имя_ветки
```

- Удаление ветки:

- удаление локальной уже слитой с основным деревом ветки:

```
git branch -d имя_ветки
```

- принудительное удаление локальной ветки:

```
git branch -D имя_ветки
```

- удаление ветки с центрального репозитория:

```
git push origin :имя_ветки
```

Стандартные процедуры работы при наличии центрального репозитория

- Работа пользователя со своей веткой начинается с проверки и получения изменений из центрального репозитория (при этом в локальное дерево до начала этой процедуры не должно было вноситься изменений):

```
git checkout master  
git pull  
git checkout -b имя_ветки
```

- Затем можно вносить изменения в локальном дереве и/или ветке.
- После завершения внесения какого-то изменения в файлы и/или каталоги проекта необходимо разместить их в центральном репозитории. Для этого необходимо проверить, какие файлы изменились к текущему моменту:

```
git status
```

- При необходимости удаляем лишние файлы, которые не хотим отправлять в центральный репозиторий.
- Затем полезно просмотреть текст изменений на предмет соответствия правилам ведения чистых коммитов:

```
git diff
```

- Если какие-либо файлы не должны попасть в коммит, то помечаем только те файлы, изменения которых нужно сохранить. Для этого используем команды добавления и/или удаления с нужными опциями:

```
git add ...  
git rm ...
```

- Если нужно сохранить все изменения в текущем каталоге, то используем:

```
git add .
```

- Затем сохраняем изменения, поясняя, что было сделано:

```
git commit -am "Some commit message"
```

- Отправляем изменения в центральный репозиторий:

```
git push origin имя_ветки
```

или

```
git push
```

Работа с локальным репозиторием

- Создадим локальный репозиторий.
- Сначала сделаем предварительную конфигурацию, указав имя и email владельца репозитория:

```
git config --global user.name "Имя Фамилия"  
git config --global user.email "work@mail"
```

- Настроим utf-8 в выводе сообщений `git`:

```
git config --global quotepath false
```

- Для инициализации локального репозитория, расположенного, например, в каталоге `~/tutorial`, необходимо ввести в командной строке:

```
cd  
mkdir tutorial  
cd tutorial  
git init
```

- После это в каталоге `tutorial` появится каталог `.git`, в котором будет храниться история изменений.
- Создадим тестовый текстовый файл `hello.txt` и добавим его в локальный репозиторий:

```
echo 'hello world' > hello.txt  
git add hello.txt  
git commit -am 'Новый файл'
```

- Воспользуемся командой `status` для просмотра изменений в рабочем каталоге, сделанных с момента последней ревизии:

```
git status
```

- Во время работы над проектом так или иначе могут создаваться файлы, которые не требуется добавлять в последствии в репозиторий. Например, временные файлы, создаваемые редакторами, или объектные файлы, создаваемые компиляторами. Можно прописать шаблоны игнорируемых при добавлении в репозиторий типов файлов в файл `.gitignore` с помощью сервисов. Для этого сначала нужно получить список имеющихся шаблонов:

```
curl -L -s https://www.gitignore.io/api/list
```

- Затем скачать шаблон, например, для C и C++

```
curl -L -s https://www.gitignore.io/api/c >> .gitignore  
curl -L -s https://www.gitignore.io/api/c++ >> .gitignore
```

Работа с сервером репозиториев

- Для последующей идентификации пользователя на сервере репозиториев необходимо сгенерировать пару ключей (приватный и открытый):

```
ssh-keygen -C "Имя Фамилия <work@mail>"
```

- Ключи сохраняться в каталоге `~/.ssh/`.
- Существует несколько доступных серверов репозиториев с возможностью бесплатного размещения данных. Например, <https://github.com/>.
- Для работы с ним необходимо сначала завести на сайте <https://github.com/> учётную запись. Затем необходимо загрузить сгенерённый нами ранее открытый ключ.
- Для этого зайти на сайт <https://github.com/> под своей учётной записью и перейти в меню *GitHub setting*.
- После этого выбрать в боковом меню *GitHub setting* > SSH-ключи и нажать кнопку *Добавить ключ*. Скопировав из локальной консоли ключ в буфер обмена:

```
cat ~/.ssh/id_rsa.pub | xclip -sel clip
```

- Вставляем ключ в появившееся на сайте поле.
- После этого можно создать на сайте репозиторий, выбрав в меню , дать ему название и сделать общедоступным (публичным).
- Для загрузки репозитория из локального каталога на сервер выполняем следующие команды:


```
git remote add origin  
ssh://git@github.com/<username>/<reponame>.git  
git push -u origin master
```

- Далее на локальном компьютере можно выполнять стандартные процедуры для работы с `git` при наличии центрального репозитория.

Базовая настройка git

Первичная настройка параметров git

- Зададим имя и email владельца репозитория:

```
git config --global user.name "Name Surname"  
git config --global user.email "work@mail"
```

- Настроим utf-8 в выводе сообщений git:

```
git config --global core.quotePath false
```

- Настройте верификацию и подписание коммитов git.
- Зададим имя начальной ветки (будем называть её `master`):

```
git config --global init.defaultBranch master
```

Учёт переносов строк

- В разных операционных системах приняты разные символы для перевода строк:
 - Windows: `\r\n` (`CR` и `LF`);
 - Unix: `\n` (`LF`);
 - Mac: `\r` (`CR`).
- Посмотреть значения переносов строк в репозитории можно командой:

```
git ls-files --eol
```

1. Параметр `autocrlf`

- Настройка `core.autocrlf` предназначена для того, чтобы в главном репозитории все переводы строк текстовых файлах были одинаковы.

- Настройка `core.autocrlf` с параметрами `true` и `input` делает все переводы строк текстовых файлов в главном репозитории одинаковыми.
 - `core.autocrlf true`: конвертация `CRLF->LF` при коммите и обратно `LF->CRLF` при выгрузке кода из репозитория на файловую систему (обычно используется в Windows).
 - `core.autocrlf input`: конвертация `CRLF->LF` только при коммитах (используются в MacOS/Linux).
- Варианты конвертации

Таблица 1.: Варианты конвертации для разных значений параметра `core.autocrlf`

<code>core.autocrlf</code>	<code>false</code>	<code>input</code>	<code>true</code>
git commit	LF -> LF	LF -> LF	LF -> CRLF
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> LF	CRLF -> CRLF
git checkout	LF -> LF	LF -> LF	LF -> CRLF
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> CRLF	CRLF -> CRLF

- Установка параметра:

- Для Windows

```
git config --global core.autocrlf true
```

- Для Linux

```
git config --global core.autocrlf input
```

2. Параметр `safecrlf`

- Настройка `core.safecrlf` предназначена для проверки, является ли окончаний строк обратимым для текущей настройки `core.autocrlf`.
 - `core.safecrlf true`: запрещается необратимое преобразование `lf<->crlf`. Полезно, когда существуют бинарные файлы, похожие на текстовые файлы.
 - `core.safecrlf warn`: печать предупреждения, но коммиты с необратимым переходом принимаются.
- Установка параметра:

```
git config --global core.safecrlf warn
```

Создание ключа ssh

Общая информация

1. Алгоритмы шифрования ssh

1. Аутентификация

В SSH поддерживается четыре алгоритма аутентификации по открытым ключам:

- DSA:
 - размер ключей DSA не может превышать 1024, его следует отключить;
- RSA:
 - следует создавать ключ большого размера: 4096 бит;
- ECDSA:
 - ECDSA завязан на технологиях NIST, его следует отключить;
- Ed25519:
 - используется пока не везде.

2. Симметричные шифры

- Из 15 поддерживаемых в SSH алгоритмов симметричного шифрования, безопасными можно считать:
 - `chacha20-poly1305`;
 - `aes*-ctr`;
 - `aes*-gcm`.
- Шифры `3des-cbc` и `arcfour` потенциально уязвимы в силу использования DES и RC4.

- Шифр `cast128-cbc` применяет слишком короткий размер блока (64 бит).

3. Обмен ключами

- Применяемые в SSH методы обмена ключей DH (Diffie-Hellman) и ECDH (Elliptic Curve Diffie-Hellman) можно считать безопасными.
- Из 8 поддерживаемых в SSH протоколов обмена ключами вызывают подозрения три, основанные на рекомендациях NIST:
 - `ecdh-sha2-nistp256`;
 - `ecdh-sha2-nistp384`;
 - `ecdh-sha2-nistp521`.
- Не стоит использовать протоколы, основанные на *SHA1*.

2. Файлы ssh-ключей

- По умолчанию пользовательские ssh-ключи сохраняются в каталоге `~/.ssh` в домашнем каталоге пользователя.
- Убедитесь, что у вас ещё нет ключа.
- Файлы закрытых ключей имеют названия типа `id_<алгоритм>` (например, `id_dsa`, `id_rsa`).

- По умолчанию закрытые ключи имеют имена:

```
id_dsa
id_ecdsa
id_ed25519
id_rsa
```

- Открытые ключи имеют дополнительные расширения `.pub`.

- По умолчанию публичные ключи имеют имена:

```
id_dsa.pub
id_ecdsa.pub
id_ed25519.pub
id_rsa.pub
```

- При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав `Enter`.
- Сменить пароль на ключ можно с помощью команды:

```
ssh-keygen -p
```

Создание ключа ssh

- Ключ ssh создаётся командой:

```
ssh-keygen -t <алгоритм>
```

- Создайте ключи:

- по алгоритму *rsa* с ключём размером 4096 бит:

```
ssh-keygen -t rsa -b 4096
```

- по алгоритму *ed25519*:

```
ssh-keygen -t ed25519
```

- При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав **Enter**.
- Сменить пароль на ключ можно с помощью команды:

```
ssh-keygen -p
```

Добавление SSH-ключа в учётную запись GitHub

- Скопируйте созданный SSH-ключ в буфер обмена командой:

```
xclip -i < ~/.ssh/id_ed25519.pub
```

- Откройте настройки своего аккаунта на GitHub и перейдем в раздел **SSH and GPG keys**.
- Нажмите кнопку **ew SSH key**.
- Добавьте в поле **Title** название этого ключа, например, **ed25519@hostname**.
- Вставьте из буфера обмена в поле **Key** ключ.
- Нажмите кнопку **Add SSH key**.

Верификация коммитов с помощью *PGP*

- Как настроить PGP-подпись коммитов с помощью **gpg**.

Общая информация

- Коммиты имеют следующие свойства:
 - author (автор) — контрибьютор, выполнивший работу (указывается для справки);
 - committer (коммитер) — пользователь, который закоммитил изменения.
- Эти свойства можно переопределить при совершении коммита.
- Авторство коммита можно подделать.
- В git есть функция подписи коммитов.
- Для подписывания коммитов используется технология PGP (см. [Работа с PGP](#)).
- Подпись коммита позволяет удостовериться в том, кто является коммитером. Авторство не проверяется.

Создание ключа

- Генерируем ключ

```
gpg --full-generate-key
```

- Из предложенных опций выбираем:
 - тип *RSA and RSA*;
 - размер 4096;
 - выберите срок действия; значение по умолчанию — 0 (срок действия не истекает никогда).
- GPG запросит личную информацию, которая сохранится в ключе:
 - Имя (не менее 5 символов).
 - Адрес электронной почты.
 - При вводе email убедитесь, что он соответствует адресу, используемому на GitHub.
 - Комментарий. Можно ввести что угодно или нажать клавишу ввода, чтобы оставить это поле пустым.

Экспорт ключа

- Выводим список ключей и копируем отпечаток приватного ключа:

```
gpg --list-secret-keys --keyid-format LONG
```

- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.
- Формат строки:

```
sec    Алгоритм/Отпечаток_ключа  Дата_создания [Флаги] [Годеи_до]
      ID_ключа
```

- Экспортируем ключ в формате ASCII по его отпечатку:

```
gpg --armor --export <PGP Fingerprint>
```

Добавление PGP ключа в GitHub

- Копируем ключ и добавляем его в настройках профиля на GitHub (или GitLab).
- Скопируйте ваш сгенерированный PGP ключ в буфер обмена:

```
gpg --armor --export <PGP Fingerprint> | xclip -sel clip
```

- Перейдите в настройки GitHub (<https://github.com/settings/keys>), нажмите на кнопку *New GPG key* и вставьте полученный ключ в поле ввода.

Подписывание коммитов git

- Подпись коммитов при работе через терминал:

```
git commit -a -S -m 'your commit message'
```

- Флаг *-S* означает создание подписанного коммита. При этом может потребоваться ввод кодовой фразы, заданной при генерации GPG-ключа.

Настройка автоматических подписей коммитов git

- Используя введенный email, укажите Git применять его при подписи коммитов:

```
git config --global user.signingkey <PGP Fingerprint>
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg2)
```

Проверка коммитов в Git

- GitHub и GitLab будут показывать значок *Verified* рядом с вашими новыми коммитами.

Режим бдительности (vigilant mode)

- На GitHub есть настройка [vigilant mode](#).
- Все неподписанные коммиты будут явно помечены как *Unverified*.
- Включается это в настройках в разделе *SSH and GPG keys*. Установите метку на *Flag unsigned commits as unverified*.

Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ *SSH*.
- Создать ключ *PGP*.
- Настроить подписи git.
- Зарегистрироваться на *Github*.
- Создать локальный каталог для выполнения заданий по предмету.

Последовательность выполнения работы

Установка программного обеспечения

Установка git

- Установим *git*:

```
dnf install git
```

Установка gh

- Fedora:

```
dnf install gh
```

Базовая настройка git

- Зададим имя и email владельца репозитория:

```
git config --global user.name "Name Surname"  
git config --global user.email "work@mail"
```


- Настроим utf-8 в выводе сообщений git:

```
git config --global core.quotePath false
```

- Настройте верификацию и подписание коммитов git (см. [Верификация коммитов git с помощью GPG](#)).
- Зададим имя начальной ветки (будем называть её **master**):

```
git config --global init.defaultBranch master
```

- Параметр **autocrlf**:

```
git config --global core.autocrlf input
```

- Параметр **safecrlf**:

```
git config --global core.safecrlf warn
```

Создайте ключи *ssh*

- по алгоритму *rsa* с ключём размером 4096 бит:

```
ssh-keygen -t rsa -b 4096
```

- по алгоритму *ed25519*:

```
ssh-keygen -t ed25519
```

Создайте ключи *pgp*

- Генерируем ключ

```
gpg --full-generate-key
```

- Из предложенных опций выбираем:
 - тип *RSA and RSA*;
 - размер 4096;
 - выберите срок действия; значение по умолчанию — 0 (срок действия не истекает никогда).
- GPG запросит личную информацию, которая сохранится в ключе:

- Имя (не менее 5 символов).
- Адрес электронной почты.
 - При вводе email убедитесь, что он соответствует адресу, используемому на GitHub.
- Комментарий. Можно ввести что угодно или нажать клавишу ввода, чтобы оставить это поле пустым.

Настройка github

- Создайте учётную запись на <https://github.com>.
- Заполните основные данные на <https://github.com>.

Добавление PGP ключа в GitHub

- Выводим список ключей и копируем отпечаток приватного ключа:

```
gpg --list-secret-keys --keyid-format LONG
```

- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.
- Формат строки:

```
sec      Алгоритм/Отпечаток_ключа  Дата_создания [Флаги] [Годе_н_до]  
         ID_ключа
```

- Скопируйте ваш сгенерированный PGP ключ в буфер обмена:

```
gpg --armor --export <PGP Fingerprint> | xclip -sel clip
```

- Перейдите в настройки GitHub (<https://github.com/settings/keys>), нажмите на кнопку *New GPG key* и вставьте полученный ключ в поле ввода.

Настройка автоматических подписей коммитов git

- Используя введённый email, укажите Git применять его при подписи коммитов:

```
git config --global user.signingkey <PGP Fingerprint>  
git config --global commit.gpgsign true  
git config --global gpg.program $(which gpg2)
```

Настройка gh

- Для начала необходимо авторизоваться

```
gh auth login
```

- Утилита задаст несколько наводящих вопросов.
- Авторизоваться можно через браузер.

Шаблон для рабочего пространства

- [Рабочее пространство для лабораторной работы](#)
- Репозиторий: <https://github.com/yamadharm/course-directory-student-template>.

Создание репозитория курса на основе шаблона

- Необходимо создать шаблон рабочего пространства (см. [Рабочее пространство для лабораторной работы](#)).
- Например, для 2022–2023 учебного года и предмета «Операционные системы» (код предмета **os-intro**) создание репозитория примет следующий вид:

```
mkdir -p ~/work/study/2022-2023/"Операционные системы"  
cd ~/work/study/2022-2023/"Операционные системы"  
gh repo create study_2022-2023_os-intro --template=yamadharm/course-directory-student-template --public  
git clone --recursive git@github.com:<owner>/study_2022-2023_os-intro.git os-intro
```

Настройка каталога курса

- Перейдите в каталог курса:

```
cd ~/work/study/2022-2023/"Операционные системы"/os-intro
```

- Удалите лишние файлы:

```
rm package.json
```

- Создайте необходимые каталоги:

```
echo os-intro > COURSE  
make
```

- Отправьте файлы на сервер:

```
git add .  
git commit -am 'feat(main): make course structure'  
git push
```

Содержание отчёта

1. Титульный лист с указанием номера лабораторной работы и ФИО студента.
2. Формулировка цели работы.
3. Описание результатов выполнения задания:
 - скриншоты (снимки экрана), фиксирующие выполнение лабораторной работы;
 - листинги (исходный код) программ (если они есть);
 - результаты выполнения программ (текст или снимок экрана в зависимости от задания).
4. Выводы, согласованные с целью работы.
5. Ответы на контрольные вопросы.

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначены?
2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.
3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.
4. Опишите действия с VCS при единоличной работе с хранилищем.
5. Опишите порядок работы с общим хранилищем VCS.
6. Каковы основные задачи, решаемые инструментальным средством `git`?
7. Назовите и дайте краткую характеристику командам `git`.
8. Приведите примеры использования при работе с локальным и удалённым репозиториями.
9. Что такое и зачем могут быть нужны ветви (branches)?
10. Как и зачем можно игнорировать некоторые файлы при commit?

Последнее изменение: Вторник, 31 января 2023, 19:53

[◀ Отчёт о выполнении.
Лабораторная работа № 1](#)

Перейти на...

[Отчёт о выполнении. Лабораторная
работа № 2 ▶](#)